Agentic AI System for Automatic SQL Query Generation and Error Correction

Abhinav Manoj Nair
Department of Electronics and Communication Engineering
Amrita School of Engineering
Amrita Vishwa Vidyapeetham
Amritapuri, Kollam, Kerala, India
am.en.u4eac23003@am.students.amrita.edu

Abin Roy
Department of Electronics and Communication Engineering
Amrita School of Engineering
Amrita Vishwa Vidyapeetham
Amritapuri, Kollam, Kerala, India
am.en.u4ece23003@am.students.amrita.edu

Alok Ajith Suja
Department of Electronics and Communication Engineering
Amrita School of Engineering
Amrita Vishwa Vidyapeetham
Amritapuri, Kollam, Kerala, India
am.en.u4ece23006@am.students.amrita.edu

***Abstract*—Large language models support automatic SQL query generation via natural language, but can often give logically incorrect queries, especially with multi-table queries. These errors are likely to run successfully but provide inaccurate outcomes. The paper introduces CAV-SQL, a framework of agentic AI that combines constraint-conscious checking and an automatic remediation strategy. It works by extracting schema rules, query validation the SQL queries by looking at the Abstract Syntax Tree, and refines the wrong query iteratively provided by feedback-driven generation. The workflow proposed has logical correctness prior to execution. Experimental results report better dependability and less logical inconsistency, as compared with the base strategies.**

***Index Terms*—Text-to-SQL, Agentic AI, SQL Verification, Large Language Models, AST Parsing, LangGraph**

I. Introduction

The swift development of data-centered apps has heightened the need to have easy and effective ways to communicate with relational databases. Historically access to databases has involved the use of Structured Query Language (SQL) knowledge which restricts access by non-technical users. The text-to-SQL systems are aimed to fill the gap by transforming natural language queries into the form of executable SQL statements, allowing its users to access information without the needs to know any database query language.

The recent developments in large language models (LLMs) have boosted the work of Text-to-SQL systems considerably. These models use their excellent understanding and generation abilities of natural language to generate SQL queries directly on user input. Consequently, the systems based on the LLM have shown promising outcomes in a variety of benchmarks and in real-life applications as well. But even with these developments, there is one serious shortcoming of LLMs: they can produce syntactically correct, but logically incorrect SQL queries.

Logical mistakes in SQL queries are usually caused by inappropriate understanding of schema relations, inappropriate JOIN terms, ambiguous column reference, and missing constraints. In contrast to syntax errors (which can be spotted easily when processing or executing a query) logical errors are usually not detected because the query will be executed successfully and may still give erroneous or inaccurate results. This phenomenon is called a silent failure that is a huge problem in the implementation of effective Text-to-SQL systems in real-life situations.

Current methods deal with such problems with such techniques as schema-aware modeling, prompt engineering and multi-step reasoning. Although such a way is more precise when it comes to generation, they are always subject to the reasoning ability of LLMs and do not imply correctness. Other systems are based on the feedback of the execution or user intervention to identify and rectify default, thus creating more latency and automation loss.

To overcome these shortcomings, this paper will present CAV-SQL, an agentic and constraint-aware framework which will enhance reliability of SQL query generation. The point is to add a possible deterministic verification layer that checks SQL queries against the constraints of the database before execution. Probabilistic generation is not entirely used by the system, but logical correctness has been imposed by ensuring that tables exist, columns valid, and overlooking primary key-foreign key relationships.

Contributions:

1) A constraint-aware verification framework for SQL queries using schema-based validation.
2) An agentic workflow integrating query generation, validation, and correction.
3) An iterative correction mechanism that improves query reliability without user intervention.
4) An evaluation demonstrating improved logical correctness over baseline methods.

II. Related work

Over the years, machine learning and natural language processing technologies have driven the development of text-to-SQL systems. Initially, approaches relied on rule-based and template matching techniques, which were time-consuming and inflexible. To address this, neural semantic parsing techniques were developed, allowing automatic transformation of natural language into SQL.

A recent advancement is schema-aware models. RAT-SQL proposed relation-aware schema encoding, which models the relationships between tables, columns and query tokens to make queries generalisable across multiple databases [10]. This method significantly improved schema linking; but is preposted with complex processing tasks and needing large amounts of annotated data, making it difficult to implement in practice.

As Large Language Models (LLMs) gains popularity, recent studies have focused on harnessing the language understanding powers of LLMs for SQL generation. Methods that employ prompts, especially chain-of-thought reasoning, have been demonstrated to enhance logical correctness by producing intermediate reasoning steps prior to generating SQL queries [4], [17]. Structured planning methods also take this approach further, by breaking query generation into phases, such as identifying the schema, reasoning, and producing SQL queries, which has resulted in better performance on complex queries [19]. However, these approaches are expensive due to the added computation and highly sensitive to prompt structure and model responses.

To overcome the shortcomings of individual models, queries can be processed by multiple agents. These frameworks break the Text-to-SQL problem into modules such as schema mapping, query decomposition and reasoning. Chain-of-Query employs a reasoning framework with multiple agents to incrementally refine queries [6]. Likewise, EBA-SQL uses entity-based analysis for improved schema linking and query generation [16]. Alternatively, question categorization and agent communication is used to enhance performance [3]. While these systems enhance performance, they add to system costs and coordination complexities.

We also explore agentic AI systems for deployment. These approaches combine LLM-based reasoning with processes for generating and validating queries. The research on schema retrieval, reasoning and validation show increased usability in enterprise apps [5], [8]. But they are still largely reliant on LLM generation, and may suffer from hallucinations and logical errors, particularly in complex SQL queries.

Another key research area is SQL correction. Language model based correction approaches seek to correct a query using incorrect SQL and schema constraints [9]. Methods that account for query context through Abstract Syntax Trees (ASTs) and few-shot learning enhance SQL correction further [11]. Interactive tools like NL-EDIT enable users to receive natural language feedback to correct SQL queries [7]. These approaches enhance correction performance but often require user feedback or specific query inputs.

Retrieval-augmented approaches improve SQL generation with external knowledge, and by using examples of similar queries. These methods enhance schema and query comprehension, especially for complex databases [15]. And schema-aware reasoning methods use graph structures to capture relationships among database elements to better match natural language and SQL queries [13]. But these require significant pre-processing, storage and processing costs.

Recent survey papers reveal the need for incorporating reasoning, planning and validation in Text-to-SQL applications [1], [20]. These studies emphasise the importance of well-crafted systems that integrate several approaches to enhance system performance. Yet, one common gap in these methods is a lightweight and deterministic pre-execution method to check the correctness of generated queries. Current approaches often use probabilistic reasoning derived from language models or feedback after query execution, which may not identify logically incorrect queries that execute and produce a result, but are not correct.

Overall, while prior research has greatly improved SQL generation through schema-aware modeling, prompt-guided reasoning, multi-agent interaction and correction, there are challenges in ensuring logical correctness, reducing hallucinations, and enabling fully autonomous correction mechanisms. But there is a need to improve logical correctness, hallucination, and fully automated correction techniques. Our approach tackles this issue through the introduction of a constraint-aware verification model coupled with an iterative correction system, allowing for reliable and fully autonomous correction mechanisms for SQL generation systems.

III. PROPOSED SYSTEM AND METHODOLOGY

The proposed system uses an agentic approach to generate reliable SQL queries through a structured process of validation and iterative refinement. The architecture integrates multiple components within a unified pipeline, including schema extraction, natural language to SQL generation, constraint-aware verification, and query execution. Each component plays a specific role in ensuring the correctness and robustness of the generated queries.

The system first extracts structural information from the database schema to guide query generation and validation. A large language model translates natural language inputs into SQL queries, leveraging schema context to improve alignment with the database structure. The generated queries are then

analyzed using a constraint-aware verification mechanism that ensures compliance with schema rules, including table existence, column validity, and relational constraints.

To enhance reliability, the system uses an iterative feedback mechanism to identify and correct errors in generated queries. Queries that fail validation are refined based on structured feedback and re-evaluated until a valid query is obtained or a predefined iteration limit is reached. This agentic workflow ensures only logically consistent queries are executed, reducing errors and improving overall system accuracy and robustness.We used chinook dataset for testing the SQL generation.

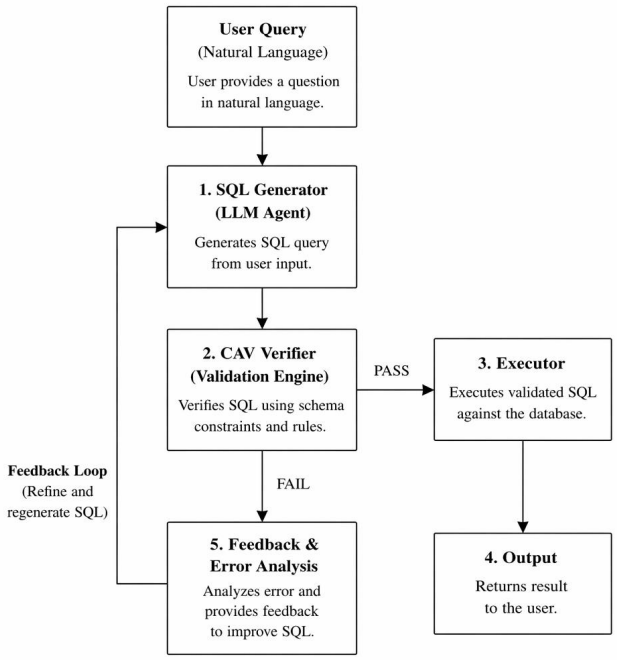

Fig. 1. Overall architecture of the proposed CAV-SQL system

Figure 1 illustrates the overall architecture of the proposed system. The process begins with a natural language query provided by the user, which is passed to a SQL generator powered by a large language model. The generated SQL query is then validated using the constraint-aware verification engine, which checks schema consistency, table existence, and join conditions.

If the query satisfies all constraints, it is forwarded to the execution module, where it is executed against the database and the result is returned to the user. If the query fails validation, the system enters a feedback loop where error analysis is performed and corrective feedback is provided to refine the query. This iterative process continues until a valid SQL query is generated or a predefined iteration limit is reached.

Algorithm 1 CAV-SQL Agentic Workflow

Require: Natural Language Question Q, Schema Rulebook S, LLM Generator G, Max Iterations M
Ensure: Executed Result Set R or Failure Exception

$iteration \leftarrow 0$
$error\_msg \leftarrow$ NULL
while $iteration < M$ **do**
  $prompt \leftarrow$ FormatPrompt$(Q, S, error_msg)$
  $draft\_sql \leftarrow G$.generate$(prompt)$
  $feedback \leftarrow$ CAV_Verify$(draft_sql, S)$ ▷ See Alg. 2
  if $feedback =$ "PASS" **then**
    $R \leftarrow$ DB_Execute$(draft_sql)$
    return R
  else
    $error\_msg \leftarrow feedback$
    $iteration \leftarrow iteration + 1$
  end if
end while
return "Execution halted: CAV validation failed."

Algorithm 2 Deterministic Constraint-Aware Verification (CAV)

Require: Draft SQL Query sql, Schema Rulebook S
Ensure: "PASS" or targeted constraint error message

$AST \leftarrow$ Parse_To_AST(sql) ▷ Build Abstract Syntax Tree
if Syntax is Invalid **then**
  return "Error: Syntax Parse Failure"
end if
$columns \leftarrow$ Extract_Columns(AST)
$joins \leftarrow$ Extract_Joins(AST)
▷ **Step 1: Extractive Validation**
for all $col \in columns$ **do**
  if $col \notin S.tables$[TableOf(col)].$columns$ **then**
    return "Error: Hallucinated column"
  end if
end for
▷ **Step 2: Semantic JOIN Guardrail**
for all $join\_clause(t_1.c_1 = t_2.c_2) \in joins$ **do**
  $valid\_fk \leftarrow$ Check_FK$(t_1.c_1 \rightarrow t_2.c_2, S)$
  $valid\_rev \leftarrow$ Check_FK$(t_2.c_2 \rightarrow t_1.c_1, S)$
  $valid\_pk \leftarrow (c_1 \in S.t_1.PKs) \wedge (c_2 \in S.t_2.PKs)$
  if $\neg(valid\_fk \vee valid\_rev \vee valid\_pk)$ **then**
    return "Error: Invalid JOIN. No schema relationship."
  end if
end for
return "PASS"

A. Schema Extraction

Our system extracts metadata from a PostgreSQL database, which includes table names, column attributes, primary keys and foreign keys. This data is then converted into a formal JSON format, called the schema rulebook. The rulebook defines the database schema in a formal way, and is used to validate constraints when executing queries.

B. SQL Query Generation

This process takes a natural language as input and uses a large language model to generate a SQL query. The model uses schema knowledge to ensure that the SQL query is compatible with the database schema. The candidate to be validated is the generated query.

C. Constraint-Aware Verification

The SQL query is verified through constraint-aware verification. The SQL is converted into an Abstract Syntax Tree (AST) for analysis. References to tables, columns and JOIN clauses are identified and verified against the rulebook.

During verification, the existence of tables, validity of columns within tables, and JOIN conditions according to primary key-foreign key relationships are checked. Any constraint violations are identified and reported with appropriate error messages.

D. Iterative Correction Mechanism

If the query fails to pass the constraints, the error feedback is given to the language model for modification. The language model optimises the query based on the violated constraints. This involves an iterative generate - validate - refine process that operates until either a valid query is found or the upper bound on the number of iterations is reached. The constraint on the number of iterations guarantees termination and efficiency.

E. Query Execution

Once the query passes the validation constraints, it is then executed. The result of the execution is obtained and returned to the user. This ensures that only syntactically correct queries are executed and incorrect queries don't return incorrect results.

F. Evaluation

This system is tested against a baseline approach which generates SQL queries without validation. Performance is measured in terms of accuracy and correctness. The experiment shows that the system enhances accuracy, especially for queries having joins of multiple tables.

IV. EXPERIMENT AND RESULTS

The system was tested to determine its ability to produce correct SQL queries based on natural language queries. Its effectiveness was compared to a baseline system that directly generates SQL queries using a large language model without a validation step.

A. Experimental Setup

We tested our approach on a PostgreSQL database with several relational tables and primary key and foreign key constraints. The experimental system was developed in Python and used a large language model to create queries. Information about the constraints was stored in a JSON file, which was used by the constraint-aware verification engine.

The system was evaluated using a range of natural language queries. These queries ranged from single-table, multi table with JOIN operations or queries with aggregate functions.

B. Evaluation Metrics

We used the performance metrics of execution accuracy, result correctness and error detection. Execution accuracy refers to the SQL query being executed. Result correctness measures whether the result of the query is correct. Avoiding incorrect queries refers to the capability of the system to detect incorrect queries before they are executed.

C. Baseline Comparison

The baseline system directly converts natural language to SQL using a large language model without any verification. While the baseline system generates queries with a high success rate, it also produces many semantically incorrect queries, especially when dealing with multiple tables and JOIN clauses.

Our system includes constraint-aware verification, which avoids execution of incorrect queries. The iterative correction mechanism also enhances the quality of the queries by successively revising incorrect ones that are validated.

D. Results and Analysis

The results show that the proposed system provides better SQL query generation. For single-table and simple queries, both the baseline and the proposed system perform similarly, churning out correct and accurate queries.

In the case of multiple table queries, the baseline system is prone to generate incorrect JOIN conditions, leading to wrong results. In the proposed system, such inconsistencies are found during the verification step and queries are not executed. The repair process allows for iterative query development and enhances correctness.

We found that, in more complicated queries, partial matches occurred due to the weakness in underlying language model and limited range of validation. But the approach monotonically decreases logical errors relative to the baseline.

E. Discussion

Our results show that adding constraint-aware verification greatly improves the success rate of Text-to-SQL systems. The use of large language models ensures that queries are syntactically correct, but not necessarily logically correct. The innovation of the proposed system is that it ensures schema validation before query execution.

The iterative correction process also enhances system performance by automatically improving queries. The deterministic validation and generative modeling approach offers a

powerful system that enables the accurate generation of SQL queries.

V. CONCLUSION AND FUTURE WORK

This paper presented CAV-SQL, an agentic AI framework for reliable SQL query generation. The system integrates constraint-aware verification with an iterative correction mechanism to ensure logical correctness before execution. By combining deterministic validation with large language model-based generation, it effectively reduces logical errors, especially in queries involving multiple tables and JOIN operations.

Experimental observations show the approach improves result correctness and prevents execution of invalid queries compared to baseline methods. The iterative correction loop further enhances performance by refining queries using structured feedback.

Despite these improvements, the system has limitations. It currently supports only basic SQL queries and does not fully handle complex constructs like nested queries and common table expressions. Future work will extend support for advanced SQL features, improve semantic validation, and optimize the correction mechanism with advanced learning techniques. Integration with larger datasets and real-world applications will also be explored to enhance scalability and robustness.

REFERENCES

[1] T. Baig, M. Usman, and S. Ahmed, "Text-to-SQL AI Agentic Workflow Implementations: A Systematic Literature Review," 2025.

[2] J. Su, Y. Li, and H. Wang, "TriSQL: A Dynamic Three-Stage Framework for Improving Text-to-SQL Query Generation," 2025.

[3] Z. Shao, L. Chen, and Y. Zhang, "Multi-Agent Collaboration with Question Classification for Text-to-SQL Generation," 2025.

[4] C. Tai, Y. Zhang, and T. Wang, "Improving Text-to-SQL with Chain-of-Thought Prompting," 2023.

[5] A. Durmusoglu, O. Sahin, and M. K. Sari, "Agentic AI System for Data-Driven Question Answering Using SQL Generation," 2024.

[6] D. Sui, Z. Chen, and Y. Li, "Chain-of-Query: Multi-Agent Collaborative Reasoning," 2025.

[7] A. Elgohary, D. Peskov, and J. Boyd-Graber, "NL-EDIT: Correcting Semantic Parsing Errors," 2021.

[8] K. Asanka and N. De Silva, "Text-to-SQL AI Agentic Workflow Implementations," 2024.

[9] Z. Chen, Y. Li, and X. Wang, "Automatic SQL Query Error Correction Using Language Models," 2023.

[10] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers," in *Proc. ACL*, 2020.

[11] R. Jain and H. Yang, "Context-Aware SQL Error Correction Using AST and Few-Shot Learning," 2024.

[12] J. Su, Y. Li, and H. Wang, "TriSQL Framework," 2025.

[13] J. Zhang, L. Wang, and T. Liu, "Schema-Aware Reasoning for Text-to-SQL Systems," 2025.

[14] Y. Huang, Q. Li, and H. Zhang, "Advanced Schema Mapping Techniques," 2024.

[15] Y. Huang, Q. Li, and H. Zhang, "Retrieval-Augmented Text-to-SQL Generation," 2024.

[16] L. Liu, H. Zhang, and Y. Chen, "EBA-SQL: Multi-Agent Collaborative Framework," 2024.

[17] J. Park, S. Kim, and H. Lee, "Interpretable Reasoning for Text-to-SQL Using Chain-of-Thought," 2023.

[18] R. Gupta et al., "Neural SQL Correction Systems," 2024.

[19] Y. Li, H. Zhao, and X. Chen, "Structured Planning for Text-to-SQL Generation," 2024.

[20] N. Deng, Y. Chen, and X. Zhang, "A Survey of Text-to-SQL in the Era of Large Language Models," 2024.